Julio Regueira Gonzalez UO309022

Sustraccion 1 tiene complejidad O(1)

Saltó stackoverflow a partir de n>8000

Sustraccion 2 tiene complejidad O(2n)

Saltó stackoverflow a partir de n>8000

Sustraccion 3 tiene complejidad O(1), (hacer regla de 3 para calcular para n=80)

Si cada vez q aumenta n una vez el tiempo aproximadamente se duplica, se tiene q duplicar desde 20 unas 60 veces. Según la complejidad O(2^(80-20), luego, 2^60. Si multiplicamos 42ms * 2^60 = 48422703193487572992ms, unos 1.535.473.845 años aproximadamente.

```
n=20**TIEMPO=42**nVeces=1
n=21**TIEMPO=82**nVeces=1
n=22**TIEMPO=168**nVeces=1
n=23**TIEMPO=331**nVeces=1
n=24**TIEMPO=652**nVeces=1
n=25**TIEMPO=1325**nVeces=1
n=26**TIEMPO=2687**nVeces=1
n=27**TIEMPO=5295**nVeces=1
n=28**TIEMPO=10740**nVeces=1
n=29**TIEMPO=21264**nVeces=1
n=30**TIEMPO=42462**nVeces=1
n=31**TIEMPO=85174**nVeces=1
n=32**TIEMPO=170598**nVeces=1
n=33**TIEMPO=339668**nVeces=1
^C
C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309
```

Sustraccion4:

```
public static void rec4 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        for (int i=0;i<n;i++){
            for (int j=0;j<n;j++){
                cont++;
            }
        }
        rec4 (n-1);
    }
}
```

n	T(ms)
100	3
200	18

400	142
800	1151
1600	9138
3200	73547

Sustraccion 5:

```
public static void rec5 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        cont++;
        rec5 (n-2);
        rec5 (n-2);
        rec5 (n-2);

    }
}
```

n	T(ms)
30	397
32	1194
34	3578
36	10705
38	32166
40	97010
42	FDT

Para n= 80?

Si la complejidad del algoritmo era O(3^(n/2)). Tomando como referencia n = 30, 397ms, Habría que multiplicar ese tiempo por un factor de conversión según la complejidad

O(3^((80-30)/2)). Por ende, 397 ms * (3^(50/2)) = 3,36 * 10^14 ms, que son unos 10654,49 años.

Division 1 tiene complejidad O(n)

Division 2 tiene complejidad O(nlogn)

Division 3 tiene complejidad O(n)

Division4:

```
public static void rec4 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        for (int i=1;i<n;i++){
            for(int j=1; j<n; j++){
                cont ++;
            }
        }
        rec4 (n/2);
    }

}
```

n	T(ms)
1000	9
2000	36
4000	137
8000	552
16000	2264
32000	9083
64000	36282
128000	FDT

Division5:

```
public static void rec5 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        cont++;
        rec5 (n/2);
        rec5 (n/2);
        rec5 (n/2);
        rec5 (n/2);
    }
}
```

n	T(ms)

1000	25
2000	104
4000	407
8000	1641
16000	6571
32000	26164
64000	104291
128000	FDT

Vector Suma y Fibonacci

```
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3>javac -d . *.java

C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3>java -Xint p3.VectorSuma
METODO1: SUMA= -687
METODO2: SUMA= -687
METODO3: SUMA= -687
METODO1 >>> n= 100  **  TIEMPO= 897 NANOSEGUNDOS
METODO2 >>> n= 100  **  TIEMPO= 2152 NANOSEGUNDOS
METODO3 >>> n= 100  **  TIEMPO= 4450 NANOSEGUNDOS

C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3>java -Xint p3.Fibonacci
METODO1: para n= 40 es= 102334155
METODO2: para n= 40 es= 102334155
METODO3: para n= 40 es= 102334155
METODO4: para n= 40 es= 102334155
METODO1 >>> n= 40**TIEMPO= 379 NANOSEGUNDOS
METODO2 >>> n= 40**TIEMPO= 566 NANOSEGUNDOS
METODO3 >>> n= 40**TIEMPO= 1003 NANOSEGUNDOS
METODO4 >>> n= 40**TIEMPO= 6111 MILISEGUNDOS
```

Para vectorSuma, todos los metodos tienen complejidad O(n). Pese a que teoricamente los 3 escalan de la misma forma linealmente, el primero va más rapido al ser un simple bucle for, el segundo sobrecarga la pila en cada paso y por eso es más lento y el tercero, pese a dividir el problema, hace dos llamadas recursivas por cada division.

Para Fibonacci los tres primeros metodos son O(n) y el cuarto es O(1,6^n). Tienen sentido los tiempos al ser el metodo 1, el iterativo, el más rapido al evitar tener que gestionar la pila de llamadas recursivas del 3 o el asignar memoria a los vectores del 2. El metodo 4 por otra parte es el más lento al recalcular los mismos problemas una y otra vez.

PuntosTrivial

```
               14 archivos        466.687 bytes
                3 dirs  85.573.332.992 bytes libres

C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3\p3p>java p3p.PuntosTrivial datos32.txt
PUNTOS MAS CERCANOS: [27,278446, 90,131051] [27,536642, 89,236468]
SU DISTANCIA MINIMA ES= 0,931098
```

```
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3\p3p>java p3p.PuntosTrivialTiempos 1
PUNTOS MAS CERCANOS: [68,494595, 48,860305] [68,481788, 48,872362]
SU DISTANCIA MINIMA ES= 0,017589
n=1024  Tiempo=29       Repeticiones=1
PUNTOS MAS CERCANOS: [84,274072, 35,816071] [84,299794, 35,794706]
SU DISTANCIA MINIMA ES= 0,033438
n=2048  Tiempo=16       Repeticiones=1
PUNTOS MAS CERCANOS: [62,322557, 65,949543] [62,313757, 65,955114]
SU DISTANCIA MINIMA ES= 0,010415
n=4096  Tiempo=20       Repeticiones=1
PUNTOS MAS CERCANOS: [11,705419, 6,174257] [11,710374, 6,152190]
SU DISTANCIA MINIMA ES= 0,022616
n=8192  Tiempo=77       Repeticiones=1
PUNTOS MAS CERCANOS: [25,590521, 83,416468] [25,593098, 83,421135]
SU DISTANCIA MINIMA ES= 0,005331
n=16384 Tiempo=306      Repeticiones=1
PUNTOS MAS CERCANOS: [90,802145, 7,732184] [90,803915, 7,733626]
SU DISTANCIA MINIMA ES= 0,002283
n=32768 Tiempo=1301     Repeticiones=1
PUNTOS MAS CERCANOS: [50,880788, 50,080815] [50,879856, 50,080577]
SU DISTANCIA MINIMA ES= 0,000962
n=65536 Tiempo=3570     Repeticiones=1
PUNTOS MAS CERCANOS: [32,564888, 83,471439] [32,564587, 83,471601]
SU DISTANCIA MINIMA ES= 0,000342
n=131072        Tiempo=14499    Repeticiones=1
PUNTOS MAS CERCANOS: [2,243394, 72,012711] [2,243499, 72,012827]
SU DISTANCIA MINIMA ES= 0,000156
n=262144        Tiempo=65714    Repeticiones=1
^C
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3\p3p>
```

n	T(ms)
1024	29
2048	16
4096	20
8192	77
16384	306
32768	1301
65536	3570
131072	14499
262144	65714

Los tiempos obtenidos se adecuan a la complejidad O(n^2) del algoritmo.

PuntosDyV

```
SU DISTANCIA MINIMA ES= 0,000156
n=262144        Tiempo=65714    Repeticiones=1
^C
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3\p3p>java p3p.PuntosDyV datos32.txt
PUNTOS MAS CERCANOS: [27,536642, 89,236468] [27,278446, 90,131051]
SU DISTANCIA MINIMA ES= 0,931098

C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3\p3p>
```

```
        at p3p.PuntosDyV.main(PuntosDyV.java:19)

C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3\p3p>java p3p.PuntosDyVTiemp
os 1
n               tiempo
------------------------
1024            13
2048            2
4096            4
8192            7
16384           28
32768           57
65536           107
131072          192
262144          186
524288          424
1048576         926
2097152         2047
4194304         4625
8388608         10256
16777216                23362
33554432                51015
^C
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3\p3p>
```

n	T(ms)
1024	13
2048	2
4096	4
8192	7
16384	28
32768	57
65536	107
131072	192
262144	186
524288	424
1048576	926
2097152	2047
4194304	4625
8388608	10256
16777216	23362
33554432	51015
67108864	FDT

Mi algoritmo tiene complejidad O(nlogn). En la tabla se observa que al duplicar el numero de puntos n, el tiempo de ejecución aumenta ligeramente más del doble.